

Values and names

Two words introduce a name: `let` for a value that never changes, and `var` for one that does.

```
use std.io

fn main() {
    let name = "Ada"
    var score = 0

    score = score + 10
    score = score + 5

    io.say("{name} scored {score}")
}
```

Reach for `let` first. Use `var` only when the value really has to change, and the next person reading your code will know which is which at a glance.

The built in types

type	what it holds	example
Int	whole numbers	42, -7, 1_000_000
Num	numbers with a decimal point	3.14, -0.5
Text	words	"hello"
Bool	true or false	true
Byte	a single byte	0x1F
Nil	nothing at all	nil

The compiler works out the type from the value, so you rarely write one down. When you want to be explicit, say so after a colon:

```
use std.io

fn main() {
    let count: Int = 3
    let ratio: Num = 0.75
    let label: Text = "widgets"
    io.say("{count} {label} at {ratio}")
}
```

Putting values into text

Curly braces inside text hold an expression. This is the usual way to build a message.

```
use std.io

fn main() {
    let width = 4
    let height = 7
    io.say("a {width} by {height} room has {width * height} square metres")
}
```

If you need a real brace, double it: `{{` prints `{` .

Numbers

`+` `-` `*` `/` do what you expect. Division always gives a `Num` , so `10 / 4` is `2.5` and never quietly throws the remainder away. `%` gives the remainder and `**` raises to a power.

```
use std.io

fn main() {
    io.say("{10 / 4}")
    io.say("{10 % 4}")
    io.say("{2 ** 10}")
}
```

Exercise

Work out how many seconds there are in a week, using named values for the parts rather than one long sum.